Document d'Architecture Technique

Billetterie des Écrans de Villeneuve

Équipe : 10

Membres : Armand LEVINE, Matteo PETRONELLI, Aurel

Version : 1.0

Date : 09/11/2026

1. Introduction et objectifs

Le produit

Les utilisateurs

Utilisateur	Ce qu'il fait avec le système	Canal
Clients	Choisissent leurs places, paient, annulent, reçoivent leurs billets	Site, appli, bornes
Caissiers	Vendent pour un client présent, encaissent	Guichets
Abonnés	Souscrivent, réservent leur place gratuite, annulent	Site, appli, guichets
Agents d'accueil	Scannent les billets	Douchettes
Programmateur / Marketing / Finance	Publient les séances, saisissent tarifs et opérations, déposent la déclaration CNC	Back-office
Partenaires (SortiesFlash, CinéMalin)	Envoient des ventes déjà payées	API

Les objectifs de qualité

Priorité	Objectif	Mesure	D'où vient-il ?
1	Cohérence de la vente : aucune place vendue deux fois	0 doublon sur 420 réservations concurrentes en 5 min (test de concurrence)	Incident de mars (14 places vendues deux fois), B1
2	Évolutivité : changer une règle ou brancher un partenaire sans toucher au cœur	Nouveau partenaire intégré en < 2 semaines, nouvelle formule d'abonnement en < 1 mois	H. Garnier, N. Ferrand
3	Tenue en charge : le pic d'avant-première ne dégrade pas la vente	Réservation répondue en < 1 s (p95) pendant le pic de 420 places en 5 min	Volumes 3.7 (avant-première, mercredi 14 h)

2. Contraintes et hypothèses

Contraintes

Code	Contrainte	Conséquence sur l'architecture
C1		
C2		
C3		
C4		
C5		
C6		

Hypothèses sur le brief

Question	Hypothèse retenue	Justification
Séance annulée : qui rembourse un billet partenaire ?	Le partenaire rembourse son client ; le cinéma l'en informe par événement et ne verse pas le reversement (tarif − 8 %) correspondant.	Le partenaire encaisse lui-même (B11) : nous n'avons ni le moyen de paiement ni la relation client.
Un abonné résilie : ses réservations du mois suivant restent-elles valables ?	Non : la résiliation prend effet fin de mois, les places gratuites réservées au-delà sont annulées.	B7 lie la place gratuite à un abonnement actif ; éviter des places gratuites sans abonnement payant.
Guichet, paiement en espèces : places bloquées 10 min ?	Non : le client est présent, la vente est conclue immédiatement à l'encaissement.	Le blocage (B2) sert à attendre un paiement distant ; au guichet l'attente n'existe pas.
Douchettes hors ligne : durée et données ?	Tenue d'une journée avec la liste des billets valides des séances du jour, synchronisée avant ouverture.	Couvre une panne pendant une avant-première (B12) ; un double scan en panne est accepté (K. Benali).

3. Contexte



Acteur ou système externe	Ce qu'il échange avec le système

4. Domaine

Glossaire

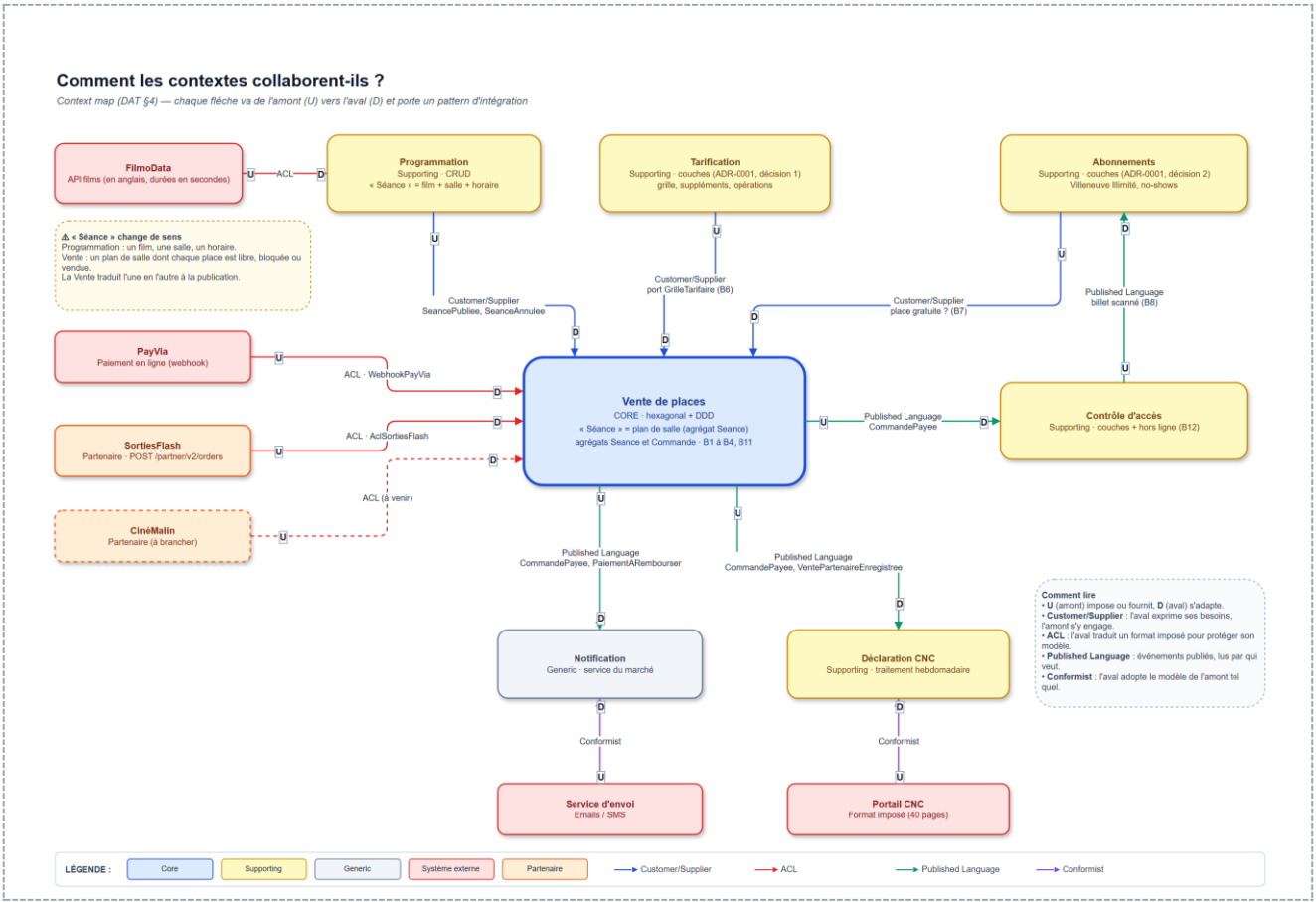
Terme	Contexte	Définition
Séance	Programmation	Un film, une salle et un horaire, publiés par le programmateur (J. Morel). Peut être annulée ou déplacée (B10).
Séance	Vente	Un plan de salle à vendre : chaque fauteuil avec son état (libre, bloqué, vendu), une heure de début et une version. Dans le code : l'agrégat Seance.
Place	Vente	Un fauteuil numéroté du plan de salle, identifié par son rang et son numéro (« G7 ») : PlaceId. Ne pas confondre avec une place vendue, qui est un billet.
Blocage	Vente	Mise de côté des places choisies pendant 10 minutes, le temps du paiement (B2). Une place bloquée n'est pas vendue ; sans paiement, elle redevient libre.
Réservation	Vente	Le geste du client qui choisit ses places : il crée une commande et bloque les places (ReserverDesPlaces). Pour un abonné, « réserver » veut dire prendre sa place gratuite (B7).
Vente	Vente	Passage définitif d'une place à l'état vendu, après paiement (Seance.vendre) ou à l'arrivée d'une vente partenaire déjà payée (B11).
Commande	Vente	L'achat d'un client : de 1 à 10 billets d'une même séance (B3), avec un statut (en attente de paiement, payée, expirée, à rembourser). Agrégat Commande.
Billet	Vente	Une place d'une commande et son tarif figé au moment du choix (B6). Après paiement, il est envoyé (QR code) et scanné au Contrôle.
Plein	Vente	Deux sens à ne pas mélanger : une séance pleine (plus aucun fauteuil libre sur le plan de salle, N. Ferrand) et le tarif plein (catégorie à 11,50 €).
Catégorie de tarif	Vente, Tarification	Plein, réduit, enfant, matin, abonné (Categorie). La Vente choisit la catégorie ; la Tarification en donne le prix.
Tarif / prix	Tarification	Le prix d'une catégorie pour une séance : grille + suppléments (3D, Grand Large), ou prix d'une opération en cours.
Opération	Tarification	Prix spécial sur une période (Fête du cinéma à 5 €) qui remplace la grille.
Expiration	Vente	Fin d'un blocage non payé : les places sont libérées et la commande passe à « expirée » (B2, ExpirerBlocages).
Paiement tardif	Vente	Paiement reçu après l'expiration (le lien PayVia vit 15 min, le blocage 10). Vente conclue si les places sont encore libres, sinon remboursement immédiat (B13).

Terme	Contexte	Définition
Abonné	Abonnements	Client titulaire d'une formule (Villeneuve Illimité) : une place gratuite par séance, deux séances à venir au maximum (B7).
No-show	Abonnements	Billet d'abonné non scanné à la fin de sa fenêtre de validité. 3 en 30 jours suspendent la réservation à l'avance 7 jours (B8).
Partenaire	Vente	Plateforme (SortiesFlash, CinéMalin) qui vend nos places à ses clients et nous envoie la vente déjà payée, dans son format (B11).
Reversement	Vente, Finance	Ce que le partenaire nous verse chaque mois : le tarif de notre grille moins 8 % (B11). Différent du montant payé par le spectateur chez le partenaire, gardé pour le CNC.
Annulation	Vente / Programmation	Deux sens : le client annule sa commande (jusqu'à 2 h avant, B9) ; le cinéma annule la séance (remboursement automatique de tous, B10).

Sous-domaines

Sous-domaine	Type	Stratégie
Vente de places	Core	Développé en interne, hexagonal + DDD. C'est là que se joue B1 (l'incident de mars : 14 places vendues deux fois) et le pic de 420 places en 5 min.
Tarification	Supporting	Développé en interne, simple : une grille et des opérations paramétrées par le marketing.
Abonnements	Supporting	Développé en interne, en couches, avec des formules paramétrées (voir ADR-0001). Stratégique pour la direction, mais sa valeur tient à l'offre commerciale, pas à un algorithme.
Programmation	Supporting	Développé en interne, CRUD : le programmateur « programme de la même façon depuis vingt ans ».
Contrôle d'accès	Supporting	Développé en interne, simple, avec un mode hors ligne (B12).
Déclaration CNC	Generic	Format imposé, identique pour tous les cinémas : on s'y conforme, et un module du marché serait envisageable.
Paiement	Generic	Acheté : PayVia (contrat de 4 ans, C3).
Notification (emails, SMS)	Generic	Délégué au service d'envoi du marché.

Context map



Amont (U)	Aval (D)	Pattern	Pourquoi
Programmation	Vente	Customer/Supplier, par événements (SeancePubliee, SeanceAnnulee)	La vente ouvre à la publication (B4). La Vente traduit la séance de la Programmation (film, salle, horaire) en sa séance (plan de salle).
Tarification	Vente	Customer/Supplier	La Vente demande le prix au moment du choix (B6) via son port GrilleTarifaire ; elle exprime ses besoins, la Tarification s'y engage.
Abonnements	Vente	Customer/Supplier	La Vente demande si l'abonné a droit à une place gratuite (B7).
PayVia	Vente	ACL (WebhookPayVia)	PayVia impose son format et ne le changera pas pour nous : on le traduit à l'entrée pour ne pas polluer le modèle de la Vente.
SortiesFlash	Vente	ACL (AclSortiesFlash)	Format imposé (annexe B) : show_ref, ADULT… restent dans adapter/in/sortiesflash/.
CinéMalin	Vente	ACL (à venir)	Même situation, autre format : une seconde ACL, rien d'autre ne change.

Amont (U)	Aval (D)	Pattern	Pourquoi
Vente	Contrôle d'accès	Published Language (événement CommandePayee)	Le Contrôle construit sa liste de billets valides à partir des ventes.
Vente	Déclaration CNC	Published Language (CommandePayee, VentePartenaireEnregistree)	Le CNC veut le prix payé par le spectateur, y compris chez les partenaires : il est dans l'événement.
Vente	Notification	Published Language (CommandePayee, PaiementARembourser)	Envoi des billets et des avis de remboursement.
Contrôle d'accès	Abonnements	Published Language (événement « billet scanné »)	Le scan prouve la venue de l'abonné : base du décompte des no-shows (B8).
FilmoData	Programmation	ACL	API en anglais, durées en secondes : traduites à l'entrée.
Portail CNC	Déclaration CNC	Conformist	40 pages de spécification, « le CNC ne négociera pas » : on adopte son modèle tel quel.
Service d'envoi	Notification	Conformist	Prestataire du marché, on suit son API.

5. Choix d'architecture par contexte

Grille de décision

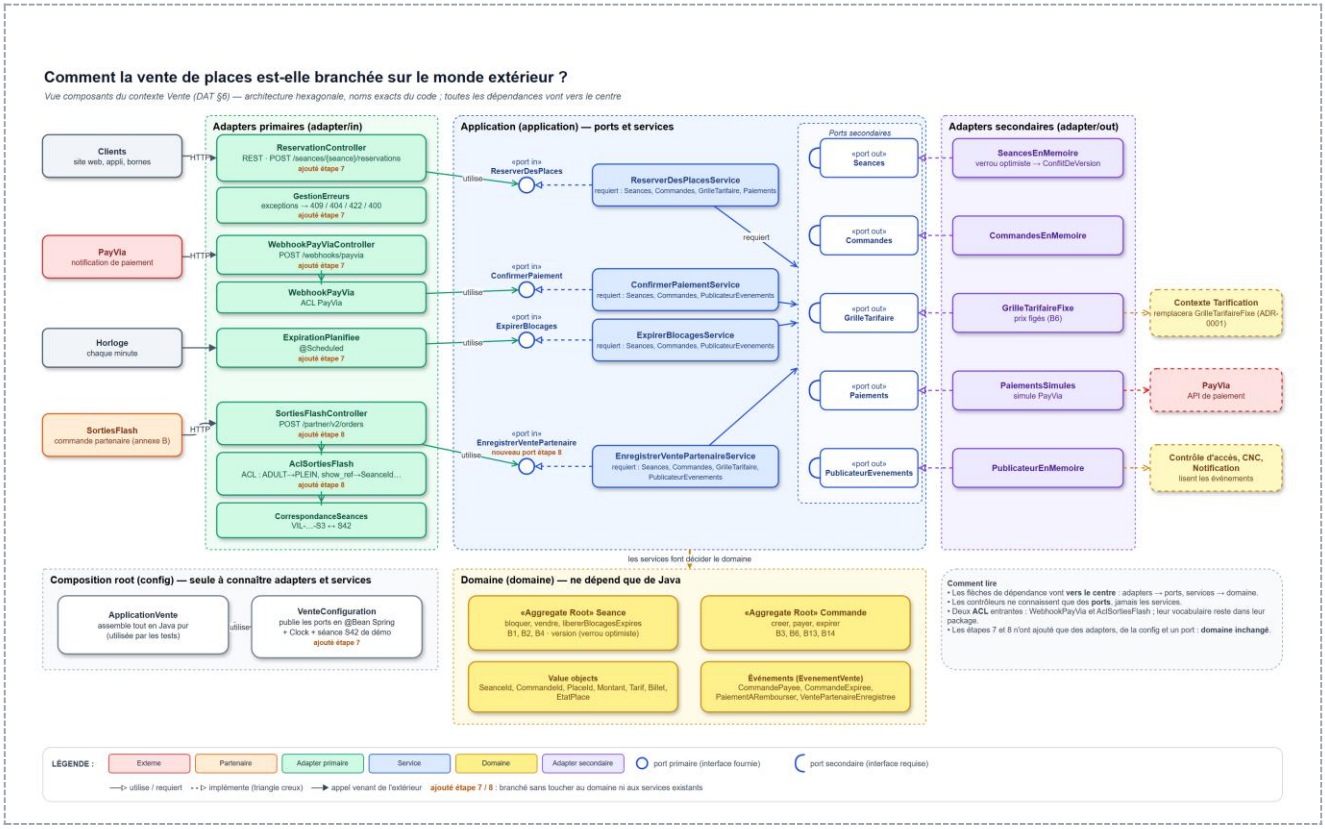
Notes de 1 (faible) à 3 (fort). « Experts » : 3 = expert disponible chaque semaine, 1 = rarement disponible. Règle de lecture : **total ≥ 14 et experts ≥ 2 → hexagonal + DDD** ; total entre 10 et 13, ou experts trop peu disponibles → architecture en couches simple ; total ≤ 9 → CRUD ou solution du marché.

Contexte	Complexité	Valeur	Durée de vie	Volatilité	Experts	Canaux	Total	Architecture retenue
Vente	3	3	3	3	3	3	18	Hexagonal + DDD
Abonnements	3	3	3	3	1	2	15	Couches, formules paramétrées
Tarification	2	2	3	3	1	1	12	Couches, grille paramétrée
Contrôle d'accès	2	2	3	1	2	1	11	Couches + mode hors ligne
Déclaration CNC	2	2	3	1	2	1	11	Couches, traitement hebdomadaire

Justification

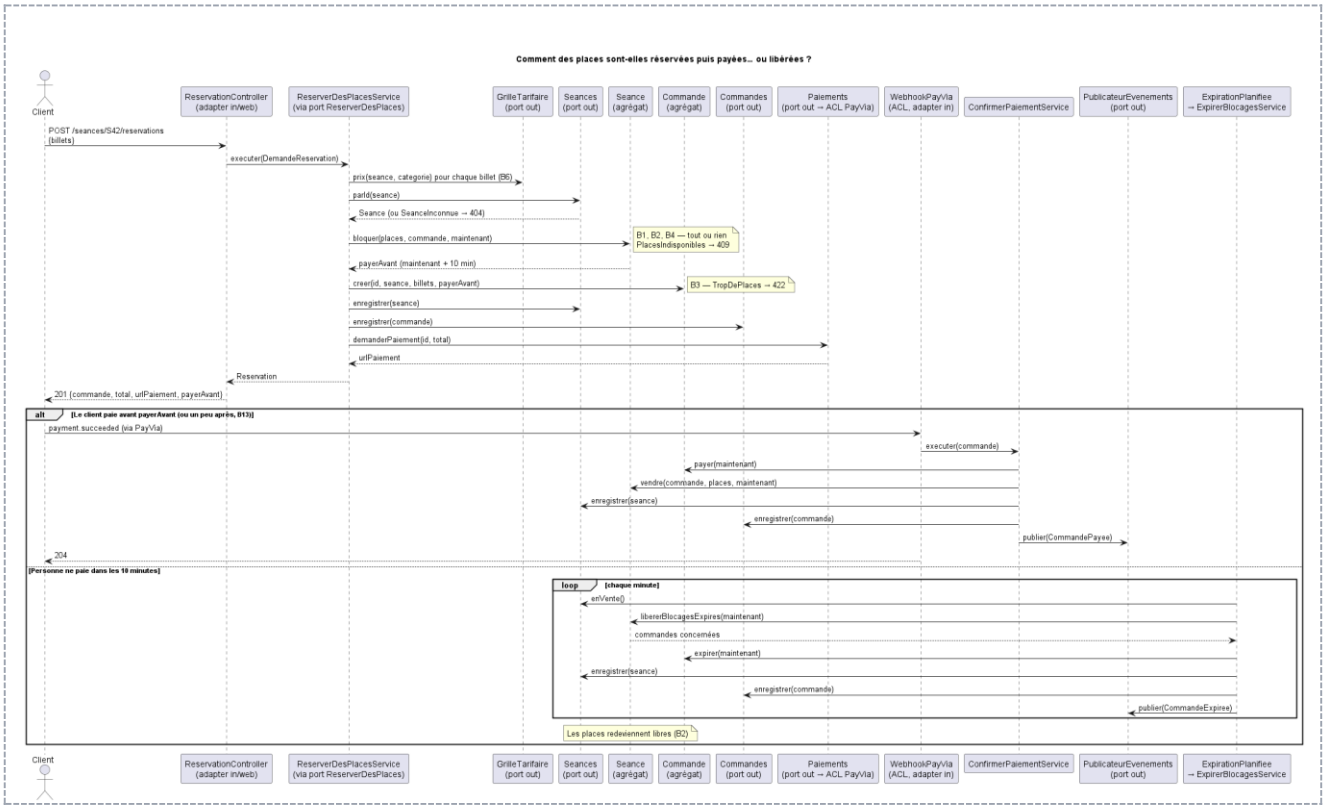
Un seul contexte justifie un DDD complet, la Vente. Les Abonnements ont un total élevé, mais le critère « experts » l'emporte : c'est le contexte que nous laissons volontairement simple, et nous l'assumons dans l'ADR-0001.

6. Vue composants

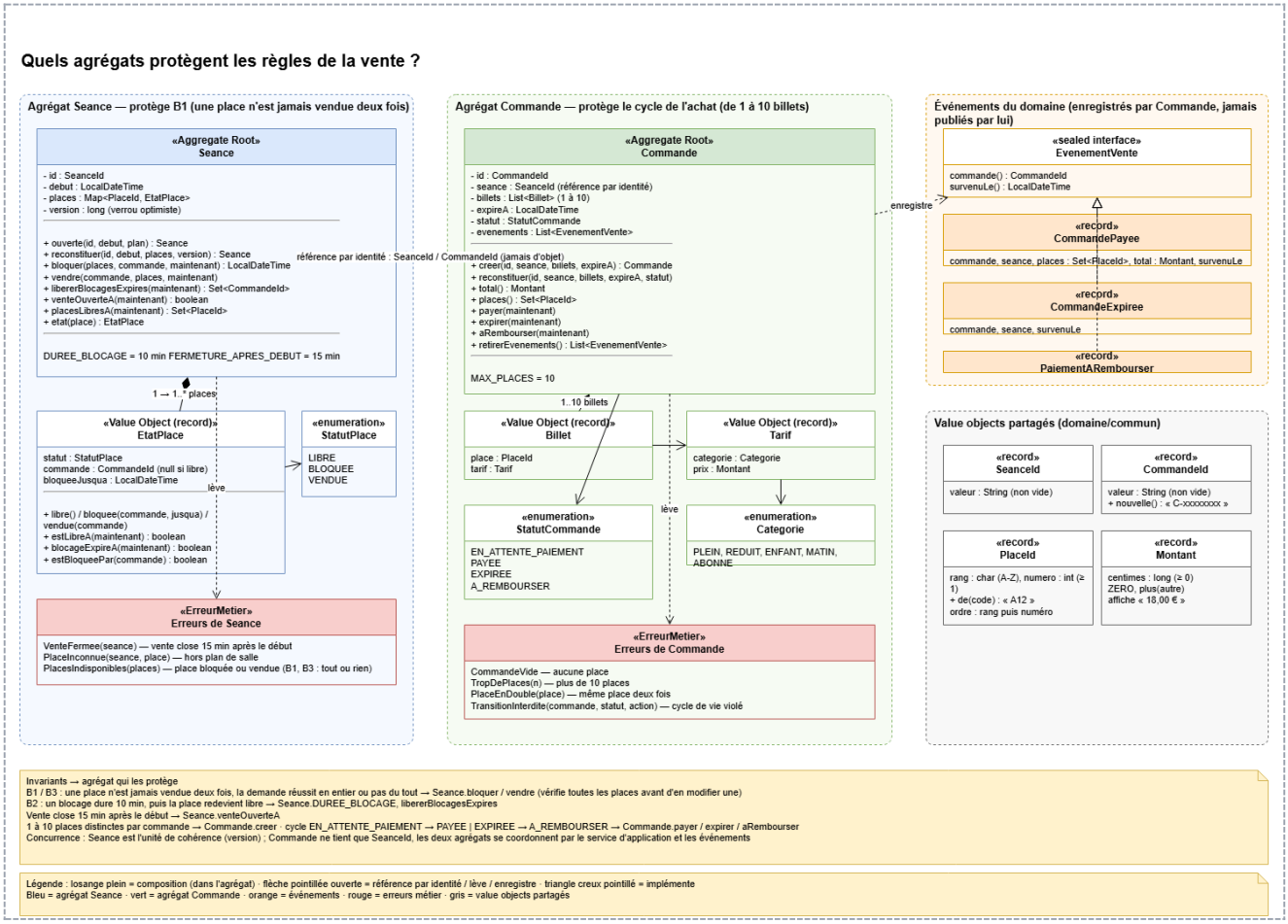


7. Vue dynamique

Réserver et payer des places



8. Modèle du domaine



Invariant (règle)	Agrégat qui le protège	Test qui le prouve
Une place n'est vendue qu'une fois (B1)	Seance	SeanceTest.refuse_une_place_deja_bloquee_par_un_autre_client_B1, refuse_une_place_vendue_B1
Réservation tout ou rien	Seance	SeanceTest.tout_ou_rien_si_une_place_est_prise_aucune_n_est_bloquee
Blocage de 10 minutes, libération automatique (B2)	Seance	SeanceTest.bloque_des_places_libres_pendant_10_minutes_B2, libere_les_blocages_expires_et_renvoie_les_commandes_concernees_B2
Vente fermée 15 min après le début (B4)	Seance	SeanceTest.la_vente_est_fermee_au_dela_B4
Paiement tardif accepté si places libres (B13)	Seance + Commande	ConfirmerPaiementTest / bonus PaiementTardifTest

Invariant (règle)	Agrégat qui le protège	Test qui le prouve
10 places maximum par commande (B3)	Commande	CommandeTest.refuse_plus_de_10_places_B3
Prix figé au moment du choix (B6)	Commande (Billet/Tarif)	CommandeTest.le_total_est_la_somme_des_prix_figes_dans_les_billets_B6
On ne paie pas deux fois, une commande payée n'expire pas	Commande	CommandeTest.on_ne_paie_pas_deux_fois, une_commande_payee_n_expire_pas

Value objects

Un value object n'a pas d'identité : deux valeurs égales sont interchangeables. Ils sont immuables (Java record) et refusent à la construction toute valeur invalide, donc un objet existant est toujours valide.

Value object	Rôle	Règle qu'il garde	Où il vit
PlaceId	Un fauteuil du plan de salle : rang (A à Z) + numéro (« G7 »)	Rang entre A et Z, numéro ≥ 1 ; ordre naturel (rang puis numéro), utile pour poser les blocages dans un ordre stable	domaine/commun
SeanceId	Identifiant d'une séance, attribué par la Programmation (« S42 »)	Non vide. La Vente ne le fabrique jamais : elle le reçoit	domaine/commun
CommandeId	Identifiant d'une commande, aussi la reference transmise à PayVia	Non vide ; généré à la création (C- + 8 caractères)	domaine/commun
Montant	Une somme en euros, stockée en centimes	Jamais négatif ; pas d'erreur d'arrondi (long, pas de décimal). Sert au prix, au total (B6) et au remboursement	domaine/commun
Categorie	Plein, réduit, enfant, matin, abonné	Ensemble fermé : la Vente choisit la catégorie, la Tarification donne le prix	domaine/commande

Événements et policies

Un événement dit ce qui s'est passé, au passé. Il est créé par l'agrégat (Commande les range dans une liste interne), puis publié par le service d'application après l'enregistrement, via le port PublicateurEvenements. Si l'enregistrement échoue, rien n'est publié. Les quatre événements sont les cas de l'interface scellée EvenementVente. Tous portent la commande concernée et l'instant survenuLe.

Événement	Émis par	Quand	Contenu	Réactions (policies)
CommandePayee	Commande.payer (via ConfirmerPaiementService)	Le paiement est confirmé et les places sont passées à « vendue »	Commande, séance, places, total	Envoyer les billets (Notification : QR code par email/SMS) · Ajouter les billets à la liste des billets valides (Contrôle d'accès) · Compter l'entrée et le prix payé (Déclaration CNC)
CommandeExpiree	Commande.expirer (via ExpirerBlocagesService, planificateur chaque minute)	Le blocage de 10 min est échu sans paiement (B2)	Commande, séance	Aucune réaction nécessaire pour la Vente : les places sont déjà libérées dans la même opération. Utile aux statistiques.
PaiementARembourser	Commande.aRembourser (via ConfirmerPaiementService)	Paiement reçu trop tard et places reprises par un autre client (B13)	Commande, montant	Rembourser le client via PayVia · Envoyer l'avis de remboursement (Notification)
VentePartenaireEnregistree	EnregistrerVentePartenaireService	Une vente déjà payée est arrivée d'un partenaire par son ACL (B11)	Commande, partenaire, référence partenaire, montant payé	Déclarer l'entrée au CNC avec le prix payé par le spectateur · Calculer le reversement dû au partenaire (tarif − 8 %) (Finance)

Policies (règles « quand X arrive, alors faire Y ») :

Policy	Déclencheur	Action	État
Libération automatique des blocages (B2)	Chaque minute, ExpirationPlanifiee	ExpirerBlocagesService libère les blocages échus et fait expirer les commandes concernées	Implémentée, testée
Paiement tardif (B13)	Notification de paiement reçue après l'expiration	Vente si les places sont encore libres, sinon Commande.aRembourser	Implémentée (ConfirmerPaiementService), testée (bonus/PaiementTardifTest)
Idempotence du paiement (B14)	Notification en double de PayVia	Ignorée si la commande est déjà PAYEE ou A_REMBOURSER	Implémentée, testée (bonus/PaiementEnDoubleTest)
Remboursement automatique	PaiementARembourser	Demander le remboursement à PayVia	À faire : l'événement est émis, mais aucun port ni adapter de remboursement n'existe encore
Envoi des billets	CommandePayee	Appel au service d'envoi	À faire : aucun consommateur branché
Billets valides du Contrôle, déclaration CNC	CommandePayee, VentePartenaireEnregistree	Alimenter les contextes aval	À faire : contextes hors du périmètre codé

Aujourd'hui le publicateur est PublicateurEnMemoire : il conserve les événements pour les tests, sans les livrer à personne. La livraison réelle reste à brancher (voir section 11).

9. Décisions (ADR)

ADR-0001 — Architecture retenue par contexte

Rubrique	Contenu
Statut	Accepté
Contexte	La billetterie compte sept contextes (section 4) pour une équipe de deux développeurs et un prestataire, qui doit la maintenir dix ans (C1). Faire du DDD partout coûterait trop cher ; n'en faire nulle part exposerait la Vente à un nouvel incident de mars. La grille (section 5) classe les contextes. Deux cas restent à trancher : la Tarification (décision 1) et les Abonnements (décision 2).
Options envisagées	Décision 1 — Tarification. A : un contexte Tarification à part, géré par le marketing, que la Vente interroge. B : le calcul du prix dans le modèle de la Vente. Décision 2 — Abonnements. A : DDD complet (agrégat Abonnement, quotas, no-shows, hexagonal). B : architecture en couches, formules paramétrées, no-shows calculés par requêtes.
Décision	Vente : hexagonal + DDD. Seul contexte au-dessus du seuil de la grille avec un expert disponible. Décision 1 : option A, Tarification à part, en Customer/Supplier : la Vente est cliente, via son port GrilleTarifaire. Les tarifs évoluent au rythme du marketing (opérations, formules en moins d'un mois), pas de la vente, et avec un autre vocabulaire (opération, supplément). Le code y est prêt : la Vente ne connaît que le port, et GrilleTarifaireFixe sera remplacé par un client du contexte Tarification sans toucher au cœur (c'est ce que l'étape 6 a corrigé). B6 reste dans la Vente : le prix obtenu est figé dans le billet au moment du choix. Décision 2 : option B, Abonnements en couches simples. Sabine Okafor n'est disponible qu'une fois par mois : le DDD se nourrit d'échanges fréquents avec l'expert. Les formules changent si souvent qu'un paramétrage (durée, prix, quota) est plus réactif qu'un modèle codé. Les no-shows (B8) sont calculés à partir des événements « billet scanné » du Contrôle. Autres contextes : couches simples ou CRUD (Programmation, Contrôle d'accès, Déclaration CNC), solution du marché (Notification, Paiement).
Conséquences positives	L'effort de modélisation se concentre là où est le risque (B1, pic de 420 places). Le marketing fait évoluer ses tarifs sans livraison de la Vente. Une nouvelle formule d'abonnement devient un paramétrage, compatible avec l'objectif « moins d'un mois ». Moins de code à maintenir pour une petite équipe.
Conséquences négatives	Tarification : une relation de plus dans la context map, et un appel à chaque réservation. Si la Tarification ne répond pas pendant une avant-première, la Vente ne peut pas fixer le prix. Parade : la Vente garde une copie locale de la grille en vigueur, rafraîchie à chaque publication du marketing. Abonnements : les règles (B7, B8, B16) risquent de se disperser dans les services et les requêtes. Avec 1 600 abonnés et +30 % par an, la complexité va croître et une refonte vers un modèle DDD deviendra probable. Nous la préparons en isolant les Abonnements derrière une interface : la Vente ne dépend que de « cet abonné a-t-il droit à une place gratuite ? ».

ADR-0002 — Agrégats et concurrence de la vente

Rubrique	Contenu
Statut	Accepté

Rubrique	Contenu
Contexte	Le problème de mars : sur une avant-première, 14 places du Grand Large (420 fauteuils) ont été vendues deux fois, et la salle était pleine en moins de 5 minutes. B1 dit qu'une place ne se vend qu'une fois, tous canaux confondus : site, appli, bornes, guichets et partenaires écrivent sur le même plan de salle. Deux questions. (1) Où passent les frontières des agrégats de la Vente ? Autrement dit, quelles règles doivent être vraies à chaque instant, et dans quel objet ? (2) Comment garantir B1 quand des centaines de ventes arrivent en même temps sur la même séance (décision 3) ? Le pic à tenir : 420 places en moins de 5 minutes, soit environ 150 à 200 commandes (2 à 3 places chacune). Elles se concentrent dans la première minute.
Options envisagées	Frontières des agrégats. • F1 : un seul gros agrégat Seance qui contient aussi les commandes. • F2 : deux agrégats, Seance (le plan de salle) et Commande (l'achat), qui se référencent par identité (SeanceId, CommandeId). • F3 : un agrégat par place. Décision 3 — concurrence. • A : un verrou optimiste sur toute la séance. Chaque enregistrement incrémente la version de la séance ; celui qui enregistre une version périmée reçoit ConflitDeVersion, recharge et réessaie (3 essais au plus). • B : un verrou par place, par exemple une ligne par place vendue avec une contrainte d'unicité (seance, place) en base.
Décision	Frontières : F2, deux agrégats. • Seance protège les règles du plan de salle : B1 (une place n'est jamais vendue deux fois), le tout ou rien d'une demande, B2 (blocage de 10 minutes) et B4 (fermeture 15 minutes après le début). Toutes ces règles portent sur l'ensemble des places de la séance : il faut les voir toutes pour décider. • Commande protège le cycle de l'achat : de 1 à 10 places distinctes (B3), le prix figé dans chaque billet (B6), et les transitions EN_ATTENTE_PAIEMENT → PAYEE / EXPIREE → A_REMBOURSER. F1 est écarté : un gros agrégat mettrait dans la même transaction des commandes qui n'ont rien à voir entre elles. F3 est écarté : avec un agrégat par place, plus personne ne voit la séance entière, et le tout ou rien devient une transaction sur plusieurs agrégats. Entorse assumée à « un agrégat par transaction » : ReserverDesPlacesService (bloquer + créer) et ConfirmerPaiementService (vendre + payer) écrivent les deux agrégats dans la même transaction. « Payée ⇒ vendue » doit être immédiat. Sinon, entre les deux écritures, le blocage pourrait expirer et la place être revendue. Les autres effets passent par des événements (CommandePayee, CommandeExpiree, PaiementARembourser). Décision 3 : option A, verrou optimiste sur la séance, avec réessai automatique (ReserverDesPlacesService.ESSAIS_MAX = 3). Nous la gardons en production, sur PostgreSQL (C1), avec une colonne version (@Version JPA, UPDATE … WHERE version = ?).

Rubrique	Contenu
Pourquoi A	• B1 est garanti par construction, dans le domaine. Seance.bloquer et Seance.vendre regardent toutes les places avant d'en modifier une. Le verrou garantit qu'elles le font sur l'état le plus récent. La règle se lit dans Seance.java et se teste sans base (SeanceTest). • Le tout ou rien est gratuit. Une demande de 4 places est une seule écriture de la séance. Avec l'option B, il faudrait poser 4 verrous, dans un ordre fixe pour éviter les interblocages, et défaire les premiers si le troisième échoue. • Le conflit coûte peu. On recharge la séance et le domaine redécide. Si les places sont différentes, le second essai réussit (ConcurrenceTest.places_differentes_le_service_reessaie_et_reussit). Si c'est la même place, c'est exactement le cas de mars : PlacesIndisponibles, 409 (meme_place_le_second_essai_constate_qu_elle_est_prise). • Le volume reste modeste. Ordre de grandeur, en supposant 200 commandes concentrées dans la première minute (~3,3 par seconde) et une transaction de ~30 ms : ~10 % des enregistrements entrent en conflit au premier essai, ~1 % au deuxième, ~0,1 % au troisième. Cela fait moins d'un client sur toute l'avant-première qui reçoit un 409 pour une place que personne d'autre ne voulait. • Cohérent avec une petite équipe : pas de verrou en base à orchestrer, une seule règle à comprendre.
Conséquences positives	• B1 et le tout ou rien restent dans l'agrégat, testés en Java pur, indépendants de la base (hexagone : la base n'est qu'un adapter). • Le même mécanisme protège tous les canaux : site, bornes, guichets et partenaires (EnregistrerVentePartenaireService passe aussi par Seance.bloquer puis vendre). • Le webhook PayVia est naturellement réessayé : un ConflitDeVersion pendant ConfirmerPaiement renvoie 409, PayVia renvoie la notification une minute plus tard, et l'idempotence (B14) rend ce second passage sûr.
Conséquences négatives (ce que ça coûte)	• Faux conflits au pic : deux clients qui veulent des places différentes de la même séance se gênent. Le second recharge et réessaie, ce qui ajoute de la latence, et après 3 conflits il reçoit un 409 alors que sa place était libre. • Toutes les ventes d'une séance passent en file : le débit d'écriture par séance est limité. Ce n'est pas un problème à 35 séances par jour, mais ça le deviendrait avec beaucoup plus de ventes simultanées par séance. • Le planificateur est aussi en concurrence : ExpirerBlocagesService enregistre les séances qu'il libère. Un conflit interrompt son passage, qui reprend à la minute suivante (une place peut rester bloquée ~1 minute de plus). • Un trou dans B13, repéré en relisant le code : si un blocage a expiré et que la place est reprise par un autre client avant le passage du planificateur, la commande d'origine reste EN_ATTENTE_PAIEMENT. Le planificateur ne la retrouvera plus, car la place porte désormais une autre commande. Si le paiement arrive ensuite, aRembourser refuse la transition depuis EN_ATTENTE_PAIEMENT.
Parades	• Mesurer : test de charge de l'avant-première (objectif de qualité n°3 : p95 < 1 s), en comptant les conflits et les 409. • Si les conflits sont trop nombreux : plus d'essais avec une petite attente aléatoire ; puis, si ça ne suffit pas, passer à l'option B pour les seules séances au pic (table des places vendues avec UNIQUE(seance, place)). Le port Seances isole ce changement de l'application. • Trou de B13 : autoriser Commande.aRembourser depuis EN_ATTENTE_PAIEMENT, ou faire expirer la commande d'origine au moment où sa place est reprise. Un test bonus à ajouter : « blocage expiré, place reprise avant le planificateur, puis paiement ».

Rubrique	Contenu
Option écartée et pourquoi	B, verrou par place. C'est son meilleur argument : au pic, seuls deux clients qui veulent le même fauteuil entrent en conflit, exactement le cas de mars et rien d'autre. Mais Seance ne protégerait plus B1 à elle seule. La garantie descendrait dans la base, sous forme de contrainte d'unicité, hors du domaine et de ses tests. Le tout ou rien demanderait plusieurs verrous ordonnés, avec un risque d'interblocage. Pour notre volume, ce serait de la complexité payée d'avance. Nous gardons B comme évolution possible, mesurée et ciblée.
Ce qui le prouve dans le code	domaine/seance/Seance.java (bloquer, vendre, version) · adapter/out/memoire/SeancesEnMemoire.enregistrer (verrou optimiste) · application/port/out/ConflitDeVersion · ReserverDesPlacesService (boucle de réessai) · ConfirmerPaiementService (vendre + payer, idempotence B14, paiement tardif B13) · GestionErreurs (ConflitDeVersion → 409) · tests bonus/ConcurrenceTest, bonus/PaiementEnDoubleTest, bonus/PaiementTardifTest.

10. Qualité et tests

Niveau	Ce qu'on teste	Comment	Tests
Domaine	Invariants des agrégats (B1 à B4, B6)	JUnit pur, sans Spring ni base	etape1/SeanceTest (13), etape2/CommandeTest (11)
Application	Orchestration des cas d'usage	Adapters en mémoire, HorlogeReglable pour maîtriser le temps	etape3 à etape5 (16), etape8/EnregistrerVentePartenaireTest (7)
ACL	Traduction du format partenaire	Java pur, sans Spring	etape8/AclSortiesFlashTest (6)
API	HTTP, codes d'erreur (201, 204, 400, 404, 409, 422)	@SpringBootTest + MockMvc, application complète	etape7/ApiVenteTest (6), etape8/VentePartenaireApiTest (5)
Acceptation	Parcours complet d'une soirée	Composition root ApplicationVente	ScenarioSoireeDuJeudiTest (2)
Cas limites (bonus)	Paiement en double (B14), paiement tardif (B13), deux caisses au même instant	Adapters en mémoire	bonus/ (7)
Architecture	Sens des dépendances	Recherches de l'étape 6 (piste : ArchUnit)	—

11. Risques et dette technique

Risque ou simplification	Impact	Parade
Persistance en mémoire : SeancesEnMemoire et CommandesEnMemoire simulent la base. Le verrou optimiste est testé, mais pas sur PostgreSQL (C1)	Une panne ou un redémarrage perd tout ; rien ne prouve encore B1 sous vraie concurrence en base	Adapter JPA avec @Version (UPDATE … WHERE version = ?) et rejouer ConcurrenceTest sur PostgreSQL ; test de charge du pic (420 places en 5 min, p95 < 1 s)
Faux conflits au pic (ADR-0002) : le verrou porte sur toute la séance	Au pic d'avant-première, des clients qui veulent des places différentes se gênent ; après 3 essais, un 409 alors que la place était libre	Mesurer le taux de 409 ; augmenter les essais avec une attente aléatoire ; en dernier recours, verrou par place (UNIQUE(seance, place)) pour les séances au pic. Le port Seances isole ce changement
Trou dans B13 (ADR-0002) : blocage expiré, place reprise par un autre client avant le planificateur, puis paiement de la commande d'origine	La commande reste EN_ATTENTE_PAIEMENT, aRembourser refuse la transition : un client a payé sans billet ni remboursement	Autoriser aRembourser depuis EN_ATTENTE_PAIEMENT, ou faire expirer la commande quand sa place est reprise ; ajouter le test bonus correspondant
Événements non livrés : PublicateurEnMemoire ne les transmet à personne	Billets non envoyés, remboursements non faits, CNC et Contrôle non alimentés, même si la vente est conclue	Brancher de vrais consommateurs ; pour ne pas perdre un événement entre l'enregistrement et la publication, passer à une table « outbox » écrite dans la transaction
Paiement simulé : PaiementsSimules et WebhookPayVia sont testés sur le format de l'annexe A, sans appel réel à PayVia	Écarts possibles à l'intégration (signature, délais, réessais 1 min / 5 min / 30 min / 2 h)	Test de contrat avec le bac à sable PayVia avant l'ouverture ; PayVia est un contrat de 4 ans (C3)
Entorse « un agrégat par transaction » (ADR-0002) : réserver et confirmer écrivent Seance et Commande ensemble	Les deux agrégats sont couplés ; la transaction s'allonge, et le jour où une partie sort de la base unique, la garantie « payée ⇒ vendue » est à repenser	Assumée : la cohérence immédiate prime. À réexaminer si la Vente est un jour découpée
Planificateur et expiration : ExpirerBlocagesService passe chaque minute et peut perdre contre une vente en cours	Une place peut rester bloquée environ 1 minute de plus que prévu (B2)	Le blocage échu ne retient déjà plus la place à la lecture (EtatPlace.estLibreA) : le retard n'empêche pas de la revendre
Tarification pas encore séparée : GrilleTarifaireFixe remplace le contexte Tarification	Prix en dur ; une opération (Fête du cinéma) ou un nouveau supplément demande une livraison	Remplacer l'adapter par un client de la Tarification (le cœur ne bouge pas) ; garder une copie locale de la grille, au cas où la Tarification ne répond pas au pic (ADR-0001)

Risque ou simplification	Impact	Parade
Abonnements en couches simples (ADR-0001)	Avec 1 600 abonnés et +30 % par an, les règles B7, B8, B16 risquent de se disperser ; une refonte vers un vrai modèle devient probable	Isoler les Abonnements derrière une interface (« cet abonné a-t-il droit à une place gratuite ? ») pour pouvoir les refondre sans toucher à la Vente
Seconde ACL partenaire (CinéMalin) non faite	Le délai « nouveau partenaire en moins de 2 semaines » n'est vérifié qu'avec SortiesFlash	Écrire l'ACL CinéMalin quand son format arrive ; seule cette ACL change, comme pour SortiesFlash à l'étape 8
Hypothèses non validées (section 2) : remboursement des billets partenaires, résiliation d'un abonné, vente au guichet, mode hors ligne	Une règle supposée à tort donne un comportement faux en production	Valider chacune avec le référent concerné avant la mise en service (le double scan hors ligne est déjà accepté par K. Benali)
Contraintes C1 à C6 non rédigées (section 2)	Contraintes d'exploitation et de budget non traduites en exigences vérifiables	Compléter avec l'équipe
Équipe de deux développeurs et durée de vie de 10 ans	Dépendance à deux personnes ; le logiciel de 2011 s'arrête fin 2027, la migration est datée	Garder le cœur testé sans framework (domaine en Java pur), documenter les décisions en ADR, éviter les dépendances qui vieillissent mal